Assignment-7

—----------

Name: Soham Chavan
PRN: 124B1F022
Class: SY IT A
Date: 07/04/2026

—----------

Aim: No SQL: Design and Develop Queries using CRUD operations. (Use CRUD operations, SAVE method, logical operators, joins etc.).

Introduction to MongoDB & NoSQL

Theory: MongoDB is a document-oriented NoSQL database that stores data as flexible JSON-like BSON documents instead of rows and tables. Collections group related documents. CRUD stands for Create, Read, Update, and Delete — the four fundamental operations. MongoDB also provides the save() method, logical query operators ($and, $or, $not, $nor), and the $lookup aggregation stage to perform SQL-style joins between collections.

CRUD Operations

Operation	MongoDB Method	Description
Create	**insertOne() / insertMany()**	Insert one or multiple documents into a collection
Create (save)	**save()**	Insert new or replace existing document by _id
Read	**find() / findOne()**	Query documents with optional filter & projection
Update	**updateOne() / updateMany()**	Modify existing documents using update operators
Delete	**deleteOne() / deleteMany()**	Remove one or multiple matching documents
Join	**aggregate + $lookup**	Merge documents from another collection (like SQL JOIN)
Logical	**$and, $or, $not, $nor**	Combine multiple query conditions

DATABASE 1: airlinedb — Airline Management System

1.1 Sample Data Setup — insertMany()

Theory: insertMany() inserts an array of documents into a collection in a single operation. MongoDB auto-generates a unique _id (ObjectId) for each document if one is not provided.

▶ Code

```
use airlinedb

// --- Airlines ---
db.airlines.insertMany([
  { _id:1, airline_name:'IndiGo',   country:'India', iata_code:'6E' },
  { _id:2, airline_name:'Air India',country:'India', iata_code:'AI' },
  { _id:3, airline_name:'SpiceJet', country:'India', iata_code:'SG' }
])

// --- Passengers ---
db.passengers.insertMany([
  { _id:1, first_name:'Arjun', last_name:'Sharma',
    email:'arjun@mail.com', phone:'9876543210',
    date_of_birth: new Date('1990-05-12'), passport_no:'P1001' },
  { _id:2, first_name:'Priya', last_name:'Verma',
    email:'priya@mail.com', phone:'9123456789',
    date_of_birth: new Date('1995-08-22'), passport_no:'P1002' },
  { _id:3, first_name:'Rahul', last_name:'Singh',
    email:'rahul@mail.com', phone:'9988776655',
    date_of_birth: new Date('1988-03-15'), passport_no:'P1003' }
])

// --- Flights ---
db.flights.insertMany([
  { _id:1, airline_id:1, flight_number:'6E-101',
    origin:'Mumbai',  destination:'Delhi',
    departure: new Date('2024-12-01T06:00:00'),
    arrival:   new Date('2024-12-01T08:00:00'),
    total_seats:180, available_seats:45, status:'Scheduled' },
  { _id:2, airline_id:2, flight_number:'AI-202',
    origin:'Delhi', destination:'Bangalore',
    departure: new Date('2024-12-01T09:00:00'),
    arrival:   new Date('2024-12-01T11:30:00'),
    total_seats:200, available_seats:0,  status:'Scheduled' },
  { _id:3, airline_id:3, flight_number:'SG-303',
    origin:'Chennai', destination:'Kolkata',
    departure: new Date('2024-12-02T07:00:00'),
    arrival:   new Date('2024-12-02T09:30:00'),
    total_seats:150, available_seats:20, status:'Delayed' }
])

// --- Bookings ---
db.bookings.insertMany([
  { _id:1, passenger_id:1, flight_id:1, seat:'12A',
    class:'Economy',  status:'Confirmed',
    booked_on: new Date('2024-11-20') },
  { _id:2, passenger_id:2, flight_id:2, seat:'3B',
    class:'Business', status:'Confirmed',
    booked_on: new Date('2024-11-21') },
  { _id:3, passenger_id:3, flight_id:1, seat:'7C',
    class:'Economy',  status:'Pending',
    booked_on: new Date('2024-11-22') }
])

// --- Payments ---
db.payments.insertMany([
  { _id:1, booking_id:1, amount:4500,  method:'UPI',         status:'Success'  },
```

```
  { _id:2, booking_id:2, amount:12000, method:'Credit Card', status:'Success'  },
  { _id:3, booking_id:3, amount:4200,  method:'Debit Card',  status:'Failed'   }
])
```

▶ **Output**

```
{ acknowledged: true, insertedCount: 3 }   // airlines
{ acknowledged: true, insertedCount: 3 }   // passengers
{ acknowledged: true, insertedCount: 3 }   // flights
{ acknowledged: true, insertedCount: 3 }   // bookings
{ acknowledged: true, insertedCount: 3 }   // payments
```

1.2 save() Method — Insert or Replace

Theory: The save() method works like an upsert: if the document has an _id that already exists it replaces the document; if no matching _id exists it inserts a new document. It is useful for idempotent write operations.

▶ **Code**

```
// Insert new passenger using save()
db.passengers.save({
  _id: 4,
  first_name: 'Sneha', last_name: 'Patel',
  email: 'sneha@mail.com', phone: '9001122334',
  date_of_birth: new Date('2000-11-30'), passport_no: 'P1004'
})

// Replace existing passenger _id:4 (update all fields)
db.passengers.save({
  _id: 4,
  first_name: 'Sneha', last_name: 'Patel',
  email: 'sneha.patel@mail.com',   // updated email
  phone: '9001122334',
  date_of_birth: new Date('2000-11-30'), passport_no: 'P1004'
})

db.passengers.findOne({ _id: 4 })
```

▶ **Output**

```
// After save() insert:
{ acknowledged: true, insertedId: 4 }

// After save() replace:
{
  _id: 4, first_name: 'Sneha', last_name: 'Patel',
  email: 'sneha.patel@mail.com', phone: '9001122334',
  passport_no: 'P1004'
}
```

1.3 READ Queries — find() and findOne()

Theory: find() returns a cursor of all matching documents. findOne() returns the first match. A projection (second argument) specifies which fields to include (1) or exclude (0). The _id field is included by default.

▶ **Code**

```
// R1: Find all flights
db.flights.find()

// R2: Find all Scheduled flights (field filter)
db.flights.find({ status: 'Scheduled' })

// R3: Find flights with available seats (comparison operator)
db.flights.find({ available_seats: { $gt: 0 } },
                { flight_number:1, origin:1, destination:1,
                  available_seats:1, _id:0 })
```

```
// R4: findOne — get one confirmed booking
db.bookings.findOne({ status: 'Confirmed' })

// R5: Count total passengers
db.passengers.countDocuments()
```

▶ Output

```
// R3 Output:
{ flight_number:'6E-101', origin:'Mumbai',  destination:'Delhi',     available_seats:45 }
{ flight_number:'SG-303', origin:'Chennai', destination:'Kolkata',   available_seats:20 }

// R4 Output:
{ _id:1, passenger_id:1, flight_id:1, seat:'12A',
  class:'Economy', status:'Confirmed', booked_on: 2024-11-20 }

// R5 Output:
4
```

1.4 Logical Operators — $and, $or, $not, $nor

▶ Code

```
// $and — Economy class AND Confirmed status
db.bookings.find({
  $and: [
    { class:  'Economy'   },
    { status: 'Confirmed' }
  ]
})

// $or — flights from Mumbai OR Delayed flights
db.flights.find({
  $or: [
    { origin: 'Mumbai'  },
    { status: 'Delayed' }
  ]
}, { flight_number:1, origin:1, status:1, _id:0 })

// $not — payments that are NOT Success
db.payments.find({
  status: { $not: { $eq: 'Success' } }
}, { booking_id:1, amount:1, status:1, _id:0 })

// $nor — flights that are neither Cancelled nor Delayed
db.flights.find({
  $nor: [
    { status: 'Cancelled' },
    { status: 'Delayed'   }
  ]
}, { flight_number:1, status:1, _id:0 })
```

▶ Output

```
// $and output:
{ _id:1, class:'Economy', status:'Confirmed', flight_id:1, seat:'12A' }

// $or output:
{ flight_number:'6E-101', origin:'Mumbai',  status:'Scheduled' }
{ flight_number:'SG-303', origin:'Chennai', status:'Delayed'   }

// $not output:
{ booking_id:3, amount:4200, status:'Failed' }

// $nor output:
{ flight_number:'6E-101', status:'Scheduled' }
```

```
{ flight_number:'AI-202', status:'Scheduled' }
```

1.5 UPDATE — updateOne() and updateMany()

Theory: updateOne() modifies the first document that matches the filter. updateMany() updates all matching documents. The $set operator changes specific fields; $inc increments a numeric field; $unset removes a field.

▶ Code

```
// updateOne — confirm booking _id:3
db.bookings.updateOne(
  { _id: 3 },
  { $set: { status: 'Confirmed' } }
)

// updateOne — decrement available_seats for flight _id:1
db.flights.updateOne(
  { _id: 1 },
  { $inc: { available_seats: -1 } }
)

// updateMany — mark all SpiceJet flights (airline_id:3) as Cancelled
db.flights.updateMany(
  { airline_id: 3 },
  { $set: { status: 'Cancelled' } }
)

// Verify
db.flights.find({ airline_id:3 }, { flight_number:1, status:1, _id:0 })
```

▶ Output

```
// updateOne booking:
{ acknowledged:true, matchedCount:1, modifiedCount:1 }

// updateOne seats:
{ acknowledged:true, matchedCount:1, modifiedCount:1 }

// updateMany flights:
{ acknowledged:true, matchedCount:1, modifiedCount:1 }

// Verify:
{ flight_number:'SG-303', status:'Cancelled' }
```

1.6 DELETE — deleteOne() and deleteMany()

Theory: deleteOne() removes the first document matching the filter. deleteMany() removes all matching documents. These operations are permanent — always filter carefully before executing a delete.

▶ Code

```
// deleteOne — remove failed payment
db.payments.deleteOne({ status: 'Failed' })

// Verify payment count
db.payments.countDocuments()

// deleteMany — remove all Cancelled bookings
db.bookings.deleteMany({ status: 'Cancelled' })

// Verify remaining bookings
db.bookings.find({}, { passenger_id:1, status:1, _id:0 })
```

▶ Output

```
// deleteOne:
```

```
{ acknowledged:true, deletedCount:1 }

// Payment count after delete:
2

// deleteMany:
{ acknowledged:true, deletedCount:0 }   // no Cancelled bookings existed

// Remaining bookings:
{ passenger_id:1, status:'Confirmed' }
{ passenger_id:2, status:'Confirmed' }
{ passenger_id:3, status:'Confirmed' }
```

1.7 $lookup — Join bookings with passenger details

Theory: The $lookup aggregation stage performs a left outer join between two collections. It matches documents from a 'foreign' collection based on a field value and embeds the result as an array field in each output document.

▶ Code

```
// Join bookings -> passengers -> flights
db.bookings.aggregate([
  {
    $lookup: {
      from:         'passengers',
      localField:   'passenger_id',
      foreignField: '_id',
      as:           'passenger_info'
    }
  },
  {
    $lookup: {
      from:         'flights',
      localField:   'flight_id',
      foreignField: '_id',
      as:           'flight_info'
    }
  },
  { $unwind: '$passenger_info' },
  { $unwind: '$flight_info'   },
  {
    $project: {
      passenger_name: {
        $concat: ['$passenger_info.first_name',' ','$passenger_info.last_name']
      },
      flight:  '$flight_info.flight_number',
      route: {
        $concat: ['$flight_info.origin',' -> ','$flight_info.destination']
      },
      seat:    1,
      class:   1,
      status:  1,
      _id:     0
    }
  }
])
```

▶ Output

```
{passenger_name:'Arjun Sharma', flight:'6E-101', route:'Mumbai -> Delhi',
  seat:'12A', class:'Economy',  status:'Confirmed'}
{passenger_name:'Priya Verma',  flight:'AI-202', route:'Delhi -> Bangalore',
  seat:'3B',  class:'Business', status:'Confirmed'}
{passenger_name:'Rahul Singh',  flight:'6E-101', route:'Mumbai -> Delhi',
  seat:'7C',  class:'Economy',  status:'Confirmed'}
```

DATABASE 2: shopsphere — E-Commerce Platform

2.1 Sample Data Setup — insertMany()

Theory: In MongoDB, related data can be embedded (nested arrays/objects inside a document) or referenced (storing an ID and looking up in another collection). Orders embed their items array for locality; products reference categories by ID.

▶ Code

```
use shopsphere

// --- Users ---
db.users.insertMany([
  { _id:1, username:'alice', full_name:'Alice Roy',
    email:'alice@mail.com', phone:'9000000001',
    address:'Mumbai', is_active:true },
  { _id:2, username:'bob',   full_name:'Bob Das',
    email:'bob@mail.com',   phone:'9000000002',
    address:'Delhi',  is_active:true },
  { _id:3, username:'carol', full_name:'Carol Nair',
    email:'carol@mail.com', phone:'9000000003',
    address:'Bangalore', is_active:true }
])

// --- Categories ---
db.categories.insertMany([
  { _id:1, name:'Electronics', description:'Gadgets and devices' },
  { _id:2, name:'Clothing',    description:'Apparel and accessories' },
  { _id:3, name:'Books',       description:'Academic and fiction' }
])

// --- Products ---
db.products.insertMany([
  { _id:1, name:'Smartphone X', category_id:1,
    price:25000, stock:50, rating:4.5 },
  { _id:2, name:'Laptop Pro',   category_id:1,
    price:75000, stock:20, rating:4.7 },
  { _id:3, name:'Cotton T-Shirt', category_id:2,
    price:599,   stock:200, rating:4.2 },
  { _id:4, name:'DBMS Textbook', category_id:3,
    price:850,   stock:100, rating:4.8 }
])

// --- Orders (items embedded inside order document) ---
db.orders.insertMany([
  { _id:1, user_id:1, status:'Delivered', total:25599,
    items: [
      { product_id:1, qty:1, unit_price:25000 },
      { product_id:3, qty:1, unit_price:599   }
    ]},
  { _id:2, user_id:2, status:'Shipped', total:75000,
    items: [{ product_id:2, qty:1, unit_price:75000 }]},
  { _id:3, user_id:3, status:'Pending', total:1198,
    items: [{ product_id:3, qty:2, unit_price:599 }]},
  { _id:4, user_id:1, status:'Cancelled', total:850,
    items: [{ product_id:4, qty:1, unit_price:850 }]}
])

// --- Reviews ---
db.reviews.insertMany([
  { _id:1, product_id:1, user_id:2, rating:5, comment:'Excellent phone!'  },
  { _id:2, product_id:2, user_id:1, rating:4, comment:'Great laptop.'     },
  { _id:3, product_id:3, user_id:3, rating:4, comment:'Good quality.'     },
  { _id:4, product_id:4, user_id:1, rating:5, comment:'Best DBMS book.'   }
```

```
])
```

▶ **Output**

```
{ acknowledged: true, insertedCount: 3 }   // users
{ acknowledged: true, insertedCount: 3 }   // categories
{ acknowledged: true, insertedCount: 4 }   // products
{ acknowledged: true, insertedCount: 4 }   // orders
{ acknowledged: true, insertedCount: 4 }   // reviews
```

2.2 save() Method

Theory: save() is useful for upsert patterns in e-commerce — add a product if it does not exist, or replace its details if it does. In modern MongoDB drivers, replaceOne with upsert:true is the equivalent recommended approach.

▶ **Code**

```
// Add a new product using save()
db.products.save({
  _id: 5,
  name: 'Wireless Headphones',
  category_id: 1,
  price: 3500,
  stock: 80,
  rating: 4.3
})

// Update product _id:5 — price change
db.products.save({
  _id: 5,
  name: 'Wireless Headphones',
  category_id: 1,
  price: 3200,   // price dropped
  stock: 80,
  rating: 4.3
})

db.products.findOne({ _id: 5 })
```

▶ **Output**

```
// After insert:
{ acknowledged:true, insertedId:5 }

// After replace:
{ acknowledged:true, matchedCount:1, modifiedCount:1 }

// findOne result:
{ _id:5, name:'Wireless Headphones', category_id:1,
  price:3200, stock:80, rating:4.3 }
```

2.3 READ Queries

Theory: MongoDB's find() supports rich query conditions using comparison operators ($gt, $lt, $gte, $lte, $eq, $ne), element operators, and projection to shape the output. sort(), limit(), and skip() allow pagination.

▶ **Code**

```
// R1: All products sorted by price descending
db.products.find({}, { name:1, price:1, stock:1, _id:0 })
           .sort({ price: -1 })

// R2: Products priced between 500 and 30000
db.products.find(
  { price: { $gte: 500, $lte: 30000 } },
  { name:1, price:1, _id:0 }
```

```
)

// R3: Active users only
db.users.find({ is_active: true }, { username:1, full_name:1, _id:0 })

// R4: Orders with status NOT Cancelled
db.orders.find(
  { status: { $ne: 'Cancelled' } },
  { user_id:1, status:1, total:1, _id:0 }
)

// R5: Top 2 most expensive products
db.products.find({}, { name:1, price:1, _id:0 })
           .sort({ price:-1 }).limit(2)
```

▶ **Output**

```
// R1 sorted:
{ name:'Laptop Pro',    price:75000, stock:20  }
{ name:'Smartphone X',  price:25000, stock:50  }
{ name:'DBMS Textbook', price:850,   stock:100 }
{ name:'Cotton T-Shirt',price:599,   stock:200 }

// R2 between 500-30000:
{ name:'Smartphone X',  price:25000 }
{ name:'DBMS Textbook', price:850   }
{ name:'Cotton T-Shirt',price:599   }

// R4 not cancelled:
{ user_id:1, status:'Delivered', total:25599 }
{ user_id:2, status:'Shipped',   total:75000 }
{ user_id:3, status:'Pending',   total:1198  }
```

2.4 Logical Operators

Theory: Logical operators make queries expressive. $and is implicit when multiple conditions are in the same filter object, but explicit $and is needed when the same field appears more than once. $or selects documents matching any condition.

▶ **Code**

```
// $and — Electronics category AND price < 30000
db.products.find({
  $and: [
    { category_id: 1      },
    { price: { $lt: 30000 } }
  ]
}, { name:1, price:1, _id:0 })

// $or — products with rating >= 4.7 OR stock < 25
db.products.find({
  $or: [
    { rating: { $gte: 4.7 } },
    { stock:  { $lt:  25  } }
  ]
}, { name:1, rating:1, stock:1, _id:0 })

// $not — products NOT in Electronics (category_id != 1)
db.products.find({
  category_id: { $not: { $eq: 1 } }
}, { name:1, category_id:1, _id:0 })

// $nor — orders that are neither Delivered nor Cancelled
db.orders.find({
  $nor: [
    { status: 'Delivered' },
    { status: 'Cancelled' }
```

```
  ]
}, { user_id:1, status:1, total:1, _id:0 })
```

▶ **Output**

```
// $and (Electronics < 30000):
{ name:'Smartphone X', price:25000 }

// $or (high rating OR low stock):
{ name:'Laptop Pro',  rating:4.7, stock:20 }
{ name:'DBMS Textbook',rating:4.8,stock:100}

// $not (not Electronics):
{ name:'Cotton T-Shirt', category_id:2 }
{ name:'DBMS Textbook',  category_id:3 }

// $nor (not Delivered, not Cancelled):
{ user_id:2, status:'Shipped',  total:75000 }
{ user_id:3, status:'Pending',  total:1198  }
```

2.5 UPDATE Operations

Theory: MongoDB update operators include $set (assign a value), $inc (increment/decrement), $push (add to array), $pull (remove from array), and $unset (remove a field). updateOne targets a single document; updateMany targets all matches.

▶ **Code**

```
// updateOne — change order status to Delivered
db.orders.updateOne(
  { _id: 2 },
  { $set: { status: 'Delivered' } }
)

// updateOne — reduce Smartphone X stock by 5
db.products.updateOne(
  { _id: 1 },
  { $inc: { stock: -5 } }
)

// updateOne — add a tag array to a product using $set
db.products.updateOne(
  { _id: 4 },
  { $set: { tags: ['education','reference','computer-science'] } }
)

// updateMany — deactivate all users from Delhi
db.users.updateMany(
  { address: 'Delhi' },
  { $set: { is_active: false } }
)

// Verify
db.products.findOne({ _id:1 }, { name:1, stock:1, _id:0 })
db.products.findOne({ _id:4 }, { name:1, tags:1, _id:0 })
```

▶ **Output**

```
// updateOne order:
{ acknowledged:true, matchedCount:1, modifiedCount:1 }

// updateOne stock:
{ acknowledged:true, matchedCount:1, modifiedCount:1 }

// updateMany users:
{ acknowledged:true, matchedCount:1, modifiedCount:1 }
```

```
// Verify product stock:
{ name:'Smartphone X', stock:45 }

// Verify tags:
{ name:'DBMS Textbook', tags:['education','reference','computer-science'] }
```

2.6 DELETE Operations

▶ Code

```
// deleteOne — remove the cancelled order
db.orders.deleteOne({ status: 'Cancelled' })

// deleteMany — remove all reviews with rating < 3
db.reviews.deleteMany({ rating: { $lt: 3 } })

// Verify
db.orders.countDocuments()
db.reviews.find({}, { product_id:1, rating:1, _id:0 })
```

▶ Output

```
// deleteOne:
{ acknowledged:true, deletedCount:1 }

// deleteMany (no review had rating < 3):
{ acknowledged:true, deletedCount:0 }

// Order count:
3

// Remaining reviews:
{ product_id:1, rating:5 }
{ product_id:2, rating:4 }
{ product_id:3, rating:4 }
{ product_id:4, rating:5 }
```

2.7 $lookup — Join products with categories

Theory: $lookup joins the products collection with categories using category_id. $unwind flattens the resulting array into individual fields. $project then shapes the final output — similar to a SQL SELECT with JOIN.

▶ Code

```
// Join products -> categories
db.products.aggregate([
  {
    $lookup: {
      from:         'categories',
      localField:   'category_id',
      foreignField: '_id',
      as:           'category_info'
    }
  },
  { $unwind: '$category_info' },
  {
    $project: {
      product_name:  '$name',
      category:      '$category_info.name',
      price:          1,
      stock:          1,
      rating:         1,
      _id:            0
    }
  },
  { $sort: { category:1, price:-1 } }
```

```
])

// Join reviews -> users -> products (3-way)
db.reviews.aggregate([
  { $lookup: { from:'users',    localField:'user_id',
               foreignField:'_id', as:'user_info' } },
  { $lookup: { from:'products', localField:'product_id',
               foreignField:'_id', as:'product_info' } },
  { $unwind:'$user_info' }, { $unwind:'$product_info' },
  { $project: {
      reviewer:     '$user_info.full_name',
      product:      '$product_info.name',
      rating:        1,
      comment:       1,
      _id:           0
  }}
])
```

▶ Output

```
// Products + categories:
{ product_name:'Laptop Pro',     category:'Electronics', price:75000, stock:20,
rating:4.7 }
{ product_name:'Smartphone X',   category:'Electronics', price:25000, stock:45,
rating:4.5 }
{ product_name:'DBMS Textbook',  category:'Books',       price:850,   stock:100,
rating:4.8 }
{ product_name:'Cotton T-Shirt', category:'Clothing',    price:599,   stock:200,
rating:4.2 }

// Reviews + user + product:
{ reviewer:'Bob Das',   product:'Smartphone X',  rating:5, comment:'Excellent phone!'  }
{ reviewer:'Alice Roy', product:'Laptop Pro',    rating:4, comment:'Great laptop.'      }
{ reviewer:'Carol Nair',product:'Cotton T-Shirt',rating:4, comment:'Good quality.'      }
{ reviewer:'Alice Roy', product:'DBMS Textbook', rating:5, comment:'Best DBMS book.'    }
```

2.8 Aggregation — $group for Summary Reports

Theory: The $group stage aggregates documents by a key field and applies accumulator expressions ($sum, $avg, $max, $min, $count). Combined with $sort and $project it produces powerful reporting queries in a single pipeline.

▶ Code

```
// Total revenue per user (non-cancelled orders)
db.orders.aggregate([
  { $match: { status: { $ne: 'Cancelled' } } },
  { $group: {
      _id:           '$user_id',
      total_spent:   { $sum: '$total' },
      order_count:   { $sum: 1 }
  }},
  { $sort: { total_spent: -1 } },
  {
    $lookup: {
      from: 'users', localField: '_id',
      foreignField: '_id', as: 'user_info'
    }
  },
  { $unwind: '$user_info' },
  { $project: {
      customer:     '$user_info.full_name',
      total_spent:   1,
      order_count:   1,
      _id:           0
  }}
])
```

```
// Average rating per category
db.products.aggregate([
  { $group: {
      _id:        '$category_id',
      avg_rating: { $avg: '$rating' },
      num_products: { $sum: 1 }
  }},
  { $lookup: { from:'categories', localField:'_id',
               foreignField:'_id', as:'cat' } },
  { $unwind: '$cat' },
  { $project: { category:'$cat.name',
                avg_rating:{ $round:['$avg_rating',2] },
                num_products:1, _id:0 } }
])
```

▶ Output

```
// Revenue per customer:
{ customer:'Bob Das',    total_spent:75000, order_count:1 }
{ customer:'Alice Roy',  total_spent:25599, order_count:1 }
{ customer:'Carol Nair', total_spent:1198,  order_count:1 }

// Avg rating per category:
{ category:'Books',       avg_rating:4.8,  num_products:1 }
{ category:'Electronics', avg_rating:4.6,  num_products:2 }
{ category:'Clothing',    avg_rating:4.2,  num_products:1 }
```

Summary of All Operations

The table below lists every query category demonstrated across both databases:

#	Operation	Database	Description
1	**insertMany()**	airlinedb	Insert airlines, passengers, flights, bookings, payments
2	**save()**	airlinedb	Insert new passenger; replace with updated email
3	**find() / findOne()**	airlinedb	Filter by status, seats; projection; countDocuments
4	**$and / $or / $not / $nor**	airlinedb	Logical filter combinations on bookings & flights
5	**updateOne() / updateMany()**	airlinedb	Confirm booking; decrement seats; cancel flights
6	**deleteOne() / deleteMany()**	airlinedb	Remove failed payment; remove cancelled bookings
7	**$lookup (2-way join)**	airlinedb	bookings + passengers + flights with $project
8	**insertMany()**	shopsphere	Insert users, categories, products, orders, reviews
9	**save()**	shopsphere	Insert & replace product (Wireless Headphones)
10	**find() with sort/limit**	shopsphere	Price range filter; active users; top-2 products
11	**$and / $or / $not / $nor**	shopsphere	Logical filters on products & orders
12	**updateOne() / updateMany()**	shopsphere	Status change; stock decrement; add tags array
13	**deleteOne() / deleteMany()**	shopsphere	Remove cancelled order; remove low-rated reviews
14	**$lookup (3-way join)**	shopsphere	products + categories; reviews + users + products
15	**$group aggregation**	shopsphere	Revenue per customer; avg rating per category

Conclusion

The database engine relies on a schema-less system to handle data by organizing it in a document style. This approach enables application-based data management with ease of handling changes in its structure. Moreover, the aggregation pipeline includes options such as match, group, lookup, and project, which offer a wide range of data analysis capabilities similar to SQL.

For this assignment, I performed various MongoDB core functions, including inserting records using insertOne and insertMany, finding records through find and findOne commands, as well as updating and removing records using update and delete. In addition, I made use of the and, or, and not logical operators to filter data. Finally, group was applied to generate reports, whereas lookup was utilized to merge records from several collections.

One major takeaway from this experience is that MongoDB is most efficient when handling flexible or hierarchical data structures. Embedding enhances performance among related data, whereas referencing becomes helpful when dealing with distributed data among multiple collections.